

## Exercitii — Functii cu `while` si `if` (nivel usor)

Scopul: te obisnuiesti sa scrii functii care **returneaza** o valoare (nu doar `void`) si folosesc `while` + `if` in interior. Toate exercitiile se rezolva in clasa `Functii.cs`. Sunt usoare — fiecare functie face UN singur lucru.

### Cum scrii o functie

```
public static int NumeFunctie(int parametru)
{
    int rezultat = 0;
    while (parametru > 0)
    {
        // logica aici
        parametru = parametru / 10;
    }
    return rezultat;
}
```

- `int` in fata numelui = functia returneaza un numar intreg
- `bool` in fata numelui = functia returneaza `true` sau `false`
- `return` = ce trimite functia inapoi cand se termina

### Exemplu rezolvat (deja in `Functii.cs`)

```
public static int SumaCifrelor(int numar)
{
    int suma = 0;
    while (numar > 0)
    {
        int uc = numar % 10;
        suma = suma + uc;
        numar = numar / 10;
    }
    return suma;
}
```

Ex. `SumaCifrelor(1234)` returneaza **10** ( $1+2+3+4$ ).

**Cum testezi:** in `Program.cs` scrii `Console.WriteLine(Functii.NumeFunctie(123));` si dai Run. Vezi numarul afisat in consola — corespunde cu exemplul?

### A. Cifrele unui numar — returneaza `int`

1. `public static int NumarCifre(int n)` — cate cifre are `n`.  
Ex. `NumarCifre(1234) => 4`. `NumarCifre(7) => 1`. `NumarCifre(100) => 3`.
2. `public static int UltimaCifra(int n)` — ultima cifra a lui `n`. (NU ai nevoie de `while` — doar `n % 10`.)  
Ex. `UltimaCifra(1234) => 4`. `UltimaCifra(7) => 7`.
3. `public static int ProdusCifrelor(int n)` — produsul cifrelor lui `n`. (Ca exemplul, dar cu `*` in loc de `+`. Atentie: pornesti de la 1, nu de la 0.)  
Ex. `ProdusCifrelor(234) => 24` ( $2*3*4$ ). `ProdusCifrelor(5) => 5`.

4. `public static int NumarCifrePare(int n)` — cate cifre pare are n.  
*Ex. NumarCifrePare(1234) => 2 (2 si 4). NumarCifrePare(7531) => 0.*
5. `public static int NumarCifreImpare(int n)` — cate cifre impare are n.  
*Ex. NumarCifreImpare(1234) => 2 (1 si 3).*
6. `public static int SumaCifrelorPare(int n)` — suma cifrelor pare ale lui n.  
*Ex. SumaCifrelorPare(1234) => 6 (2+4). SumaCifrelorPare(7531) => 0.*
7. `public static int SumaCifrelorImpare(int n)` — suma cifrelor impare ale lui n.  
*Ex. SumaCifrelorImpare(1234) => 4 (1+3). SumaCifrelorImpare(2468) => 0.*
8. `public static int CifraMaxima(int n)` — cea mai mare cifra a lui n. (Hint: pornesti cu `max = 0`, in interiorul while-ului daca cifra curenta > max, atunci max = cifra.)  
*Ex. CifraMaxima(3892) => 9. CifraMaxima(1111) => 1.*
9. `public static int CifraMinima(int n)` — cea mai mica cifra a lui n. (Hint: pornesti cu `min = 9`.)  
*Ex. CifraMinima(3892) => 2. CifraMinima(1111) => 1.*
10. `public static int AparitiiCifra(int n, int c)` — de cate ori apare cifra c in n.  
*Ex. AparitiiCifra(12347747, 7) => 3. AparitiiCifra(1234, 9) => 0.*

## B. Verificari — returneaza `bool`

11. `public static bool EstePar(int n)` — true daca n e par. (1 linie, fara while.)  
*Ex. EstePar(8) => true. EstePar(7) => false.*
12. `public static bool EsteDivizibilCu(int n, int k)` — true daca n se imparte exact la k. (1 linie, fara while.)  
*Ex. EsteDivizibilCu(12, 3) => true. EsteDivizibilCu(10, 3) => false.*
13. `public static bool ContineCifra(int n, int c)` — true daca n contine cifra c.  
*Ex. ContineCifra(1234, 3) => true. ContineCifra(1234, 9) => false.*
14. `public static bool ArePatruCifre(int n)` — true daca n are exact 4 cifre. (Hint: poti folosi `NumarCifre` de la ex. 1.)  
*Ex. ArePatruCifre(1234) => true. ArePatruCifre(123) => false. ArePatruCifre(12345) => false.*

## C. Sume si numarari — recapitulare while simplu

15. `public static int SumaDeLa1LaN(int n)` — suma  $1+2+\dots+n$ . (Atentie: pornesti cu `ct = 1`, conditie `ct <= n`.)  
*Ex. SumaDeLa1LaN(5) => 15. SumaDeLa1LaN(100) => 5050.*
16. `public static int SumaPareloraPanaLaN(int n)` — suma numerelor pare  $0+2+4+\dots+n$ .  
*Ex. SumaPareloraPanaLaN(10) => 30 (2+4+6+8+10). SumaPareloraPanaLaN(5) => 6 (2+4).*
17. `public static int CateMultipliDe3(int n)` — cate numere de la 1 la n se impart la 3.  
*Ex. CateMultipliDe3(10) => 3 (3, 6, 9). CateMultipliDe3(20) => 6.*
18. `public static int Factorial(int n)` —  $n! = 1*2*3*\dots*n$ . (Atentie: pornesti cu `produs = 1`.)  
*Ex. Factorial(5) => 120. Factorial(1) => 1. Factorial(4) => 24.*

## Reguli

1. Scrii functia in `Functii.cs`.

2. O testezi din `Program.cs` cu **cel puțin 2 exemple** diferite — verifici rezultatul cu cel din exemplu.
3. Dacă funcția se blochează (programul nu se oprește), ai uitat `n = n / 10;` sau `ct++;` în `while`.
4. Dacă rezultatul e cu 1 mai mic decât ar trebui, condiția probabil e `<` în loc de `<=`.

*18 funcții în total. Fa-le pe rand, în ordine. Când termini A, treci la B, apoi C.*